

Folien-Analyse — JavaScript (Säule 3)

Quelle: js/Day1 Eigenschaften-Browser-APIs-Event-Handler-Formularzugriff · Day2 AJAX-XMLHttpRequest-Fetch-JSON-Promises-Arrow-Functions-Classes. Gegen MDN verifiziert. Folienfehler mit . Themen-IDs J1-J11.

Grundlagen (Querschnitt)

- **Eigenschaften:** interpretiert (keine Kompilierung), dynamisch typisiert, Multi-Paradigma (OO + funktional), ein Root-Objekt (`window`), ein Ausführungskontext, Sandbox (same-origin-policy, kein direkter Systemzugriff).
- **4 Einbindungen:** (1) Event-Handler-Attribut `onclick="..."`, (2) `<script>...</script>`, (3) `<script src="x.js">`, (4) Konsole/REPL.
- **Datentypen:** String (`"` / `'` / ``` -Template), Number, Boolean, `null`, `undefined`, Object, Array. Typ ergibt sich aus dem Ausdruck.

J1 — Scope, Hoisting, var/let/const, TDZ (Tracing, hoch)

Mechanismus	Regel
var	Function-Scope , Hoisting (Deklaration hochgezogen, Wert nicht → vor Zuweisung undefined)
let/const	Block-Scope , KEIN nutzbares Hoisting (TDZ → ReferenceError vor Deklaration)
const	Block-Scope + keine Neuzuweisung (TypeError)
kein Schlüsselwort	implizit globale Variable ("window trashing", schlechter Stil)

```
function hoist(){ console.log(c); var c = 1; console.log(c); } // undefined, 1
function block(){ for(let i=0;i<1;i++){ console.log(i); } // ReferenceError
```

Fallen / Bugs

- **Falle:** `var` in `for` lebt nach der Schleife weiter (Function-Scope); `let` nicht.
- **Falle:** `var x` vor Zuweisung = `undefined` (Hoisting); `let x` vor Zeile = ReferenceError (TDZ).
- **Falle:** Zuweisung ohne `let/var/const` erzeugt globale Variable.

Drill

- [Tracen] `console.log(a); var a=5;` Ausgabe?

Lösung: `undefined` (Hoisting).

- [Tracen] Innere `let b=1` neben äußerem `let b=0` — was loggt das äußere `b` ?

Lösung: `0` (lokales `b` ist eigene Variable, überschattet nur im Block).

J2 — `this`-Bindung + `new` vs. Plain Call (Tracing, hoch)

Aufrufart	<code>this</code>	Rückgabe
<code>new Foo()</code>	neues, leeres Objekt	das neue Objekt (primitiver return wird ignoriert)
<code>Foo()</code> (plain)	global/window (bzw. undefined strict)	der return-Wert zählt; <code>this.x=...</code> landet global
<code>obj.foo()</code>	<code>obj</code>	normal
Arrow <code>()=></code>	<code>this</code> des umgebenden Scopes (lexikalisch)	normal

```
function Position(x,y){ this.x=x; this.y=y; return ("x+", "y+"); }
Position(3,7); // "(3,7)" (String, this=window)
new Position(3,7); // {x:3, y:7} (Objekt, return-String ignoriert)
```

Fallen / Bugs

- **Falle:** `new` ignoriert primitiven `return` und liefert das Objekt; Plain-Call liefert den `return`.
- **Falle (klassisch):** in einem Callback (z.B. `.then(function(){...})`) ist `this` NICHT das äußere Objekt → `undefined`. Lösung: `var that=this;` ODER **Arrow-Function** (lexikalisches `this`).
- FOLIE: `console.log(window.a)` → Tippfehler, korrekt `window.a`.

Drill

- [Tracen] `let a = new Calculator("Tolle", c)` mit `return zahl;` im Konstruktor — was ist `a`?

Lösung: das Objekt (Properties via `this`), der `return zahl` wird verworfen.

- [MC] Welche behält das äußere `this` im Callback: `function(){}` oder `()=>{}?`

Lösung: Arrow-Function.

J3 — Funktionen, Parameter, Rückgabewerte (Tracing, hoch)

- **Deklaration:** `function name(p1,p2){ return wert; }`.
- **Parameteranzahl beliebig:** fehlende Parameter = `undefined`; überzählige via `arguments`-Objekt (`fn.arguments[i]`, Hilfskonstrukt).
- **Kein/leeres return** → implizit `undefined`.
- **Deklaration muss** im selben oder textuell vorherigen Skript-Block stehen (function declarations werden gehoiested).

```
function func(p1,p2){ return p1; }
func("eins","zwei"); // p1="eins", p2="zwei", return "eins"
func("drei"); // p1="drei", p2=undefined, return "drei"
```

Konstruktor-Muster

Mit einer Konstruktor-Funktion und `new` lassen sich mehrere gleichartige Objekte erzeugen:

```
function Calculator(x,y){ this.x=x; this.y=y; this.sum=function(){return this.x+this.y;}; }
const c = new Calculator(4,3); c.sum(); // 7
// Objekt-Literal:
const z = { x:2, y:3, sum:function(){ return this.x+this.y; } }; z.sum(); // 5
```

Fallen / Bugs

- FOLIE: zeigt `const c = Calculator(4,3)` (OHNE `new`) und danach `c.diff=...` — das schlägt fehl (Plain-Call gibt `undefined` zurück, kein Objekt). Korrekt nur mit `new`.

Drill

- [Tracen] `func("x")` mit 2 Parametern — Wert von `p2`?

Lösung: `undefined`.

J4 — DOM-Manipulation (Produktion, hoch)

- **DOM** = Baum-Repräsentation des HTML im Speicher; Einstieg über `document`.

Aufgabe	Methode
Suchen (modern)	<code>document.querySelector("css")</code> (erstes), <code>querySelectorAll("css")</code> (alle)

Aufgabe	Methode
Suchen (klassisch)	<code>getElementById</code> , <code>getElementsByName</code> , <code>getElementsByTagName</code>
Erzeugen	<code>document.createElement("div")</code> , <code>document.createTextNode("...")</code>
Einhängen	<code>parent.appendChild(el)</code> (erst dann sichtbar!)
Ersetzen	<code>parent.replaceChild(neu, alt)</code>
Entfernen	<code>parent.removeChild(alt)</code> FOLIE schreibt <code>replaceChild(oldElement)</code> — korrekt: <code>removeChild</code>
Attribut	<code>el.getAttribute("n")</code> , <code>el.setAttribute("n", "v")</code>
Navigation	<code>firstElementChild</code> , <code>nextElementSibling</code> , <code>parentNode</code>
Inhalt	<code>textContent</code> (alles), <code>innerText</code> (sichtbar), <code>innerHTML</code> (HTML)
Style	<code>el.style.color = "red"</code>

```
const ul = document.createElement("ul");
const li = document.createElement("li");
li.setAttribute("land", "de");
ul.appendChild(li);
document.getElementById("inhalt").appendChild(ul);
```

Fallen / Bugs

- **Falle:** `createElement` passiert nur im Speicher — sichtbar erst nach `appendChild` an den (sichtbaren) Baum.
- **Falle:** `innerHTML` = mit ungültigem/User-Input → Sicherheitsrisiko (XSS) bzw. `SyntaxError`.
- **Kontrast:** `textContent` (kompletter Text inkl. hidden) vs. `innerText` (nur sichtbar) vs. `innerHTML` (Markup).

Drill

- [Erstellen] `<li land="de">` erzeugen und in `div#inhalt` einfügen.

Lösung: siehe Code oben.

J5 — Objekte/Arrays/JSON, for...in / for...of (Tracing, mittel)

- **Arrays:** `[]`, `new Array(3)` (Länge 3), `[1,2,3]`; `.length`; Zugriff `a[i]` (Lücken erlaubt).
- **for-Varianten:**
- `for(const k in obj)` → iteriert **Schlüssel/Indizes**.
- `for(const v of iterable)` → iteriert **Werte**.
- klassisch `for(init; cond; step)`.

Fallen / Bugs

- **Falle:** `for...in` liefert Keys (bei Array Indizes als String), `for...of` Werte. Verwechslung = häufige Trace-Fälle.

Drill

- [Tracen] `for(const g in {a:1,b:2}) sum+=...` — was iteriert `g`?

Lösung: die Schlüssel `"a"`, `"b"`.

J6 — Events (addEventListener, Bubbling) (mittel)

- **Inline:** `<p onclick="alert('x')">`.
- `addEventListener(type, listener[, options|useCapture])` — `type` ohne `on` (`"click"`). `removeEventListener` analog.
- **Optionen:** `once` (1x), `capture` / `useCapture` (`true` = Capturing zuerst Eltern; `false` = Bubbling, Default).
- **Event-Objekt:** `event.target` (Auslöser), `event.type`, `event.stopPropagation()`; Subklassen `MouseEvent` (`clientX`, `ctrlKey`), `KeyboardEvent` (`key`, `code`).

Fallen / Bugs

- **Falle:** Default ist **Bubbling** (innen → außen). `useCapture:true` dreht auf Capturing (außen → innen).
- **Falle:** Type ohne `on`-Präfix bei `addEventListener` (`"click"`, nicht `"onclick"`).
- **Extra-Parameter** nur über Wrapper-Funktion: `el.addEventListener("click", e=>listener(e,p1,p2))`.

Drill

- [MC] Was loggt `event.target` ?

Lösung: das Element, das das Event ausgelöst hat.

J7 — Typkonvertierung & ==/=== (MC/Tracing, mittel)

- `==` mit automatischer Typumwandlung; `===` ohne (Wert + Typ).
- `+` mit String konkateniert; arithmetisch sonst.

```
"42" + 77    // "4277" (String-Konkatenation)
"42" * 1 + 77 // 119   (zuerst Zahl)
"43" == 43    // true
"43" === 43   // false
"43" !== 43   // true
```

- **Falsy:** `false`, `0`, `""`, `undefined`, `null`, `NaN`.
- **Truthy:** alles andere — inkl. `"0"`, `[]`, `{}`, `Infinity`, `42`.

Fallen / Bugs

- **Falle:** `[]` und `{}` sind **truthy**, aber `[] == false` ist `true` (Koersion) — `===` vermeiden solche Überraschungen.
- **Falle:** `"0"` (String) ist **truthy**, `0` (Zahl) ist **falsy**.

Drill

- [Tracen] `"5" + 3` und `"5" - 3` ?

Lösung: `"53"` (Konkat), `2` (Subtraktion erzwingt Number).

- [MC] **truthy** oder **falsy**: `[]`, `""`, `"0"`, `0` ?

Lösung: **truthy**, **falsy**, **truthy**, **falsy**.

J8 — AJAX: XMLHttpRequest vs. Fetch (Produktion, hoch)

- **AJAX:** asynchrone Datenübertragung ohne Seiten-Neuladen; Server liefert meist JSON; JS verarbeitet und aktualisiert DOM.
Nachteile: A11y, Browser-Navigation/Bookmarks, SEO.
- **APIs:** SOAP (XML über HTTP POST) vs. REST (HTTP-Methoden GET/POST/PUT/DELETE, meist JSON).

XMLHttpRequest (alt, event-basiert)

Der klassische Weg für AJAX-Anfragen - umständlicher und callback-basiert:

```
const req = new XMLHttpRequest();
req.addEventListener("load", function(){ console.log(this.responseText); });
req.open("GET", url);    // open VOR send
req.send();
```

- `readyState`: 0 erzeugt · 1 open() · 2 Header empfangen · 3 Empfangen · 4 fertig. Antwort: `responseText` / `responseXML`.

Fetch (modern, Promise-basiert)

Die moderne Alternative zu XHR, sauberer mit Promises und async/await:

```
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data));
```

- **Response-Objekt:** `status`, `statusText`, `ok` (200–299), Inhalt als Promise via `text()` / `json()` / `blob()` / `formData()`.

Fallen / Bugs

- **Kontrast:** XHR = event-basiert (`readystatechange`, `readyState==4`); Fetch = Promise-basiert (`.then`).
- **Falle:** `open()` muss VOR `send()`. Bei Fetch zwei `.then` nötig (Response → `.json()` ist selbst ein Promise).
- **Falle:** Fetch rejectet NICHT bei HTTP-Fehlern (404/500) — `response.ok` / `status` selbst prüfen.

Drill

- [Erstellen] GET-Request mit fetch, JSON parsen, in Konsole loggen.

Lösung: `fetch(url).then(r=>r.json()).then(d=>console.log(d));`

- [Tracen] Welcher `readyState` = fertig?

Lösung: `4`.

J9 — Promises, async/await, Arrow Functions (Tracing/Produktion, hoch)

- **Promise-Handler:** `.then(onResolve, onReject)`, `.catch(onError)`, `.finally()`.
- **Arrow Function:** `(x) => x+1`; **this** **lexikalisch** (aus umgebendem Scope, nicht vom Aufrufer gesetzt) → löst das klassische Callback-`this`-Problem ohne `var that=this`.

```
// Arrow erbt this aus dem Konstruktor:
this.download = function(){
  fetch(url).then((data) => { console.log(this.name); }); // korrektes this
};
```

Fallen / Bugs

- **Falle:** normale `function(){}` als Callback verliert `this`; Arrow behält es.
- **Falle:** `.then` gibt selbst ein Promise zurück → Verkettung möglich; Fehler mit `.catch` fangen.

Drill

- [Erstellen] Arrow-Funktion, die zwei Zahlen addiert.

Lösung: `const add = (a,b) => a+b;`

- [Tracen] Warum loggt `()=>this.name` im Callback korrekt, `function(){return this.name}` aber `undefined`?

Lösung: Arrow erbt `this` lexikalisch; normale Funktion bekommt eigenes (globales) `this`.

J10 — ES6-Classes vs. Konstruktor/Prototype (Definition/Produktion, mittel)

- **Prototype:** jedes Objekt hat `prototype`-Kette; lesende Zugriffe werden rekursiv weitergeleitet (Ende: `null`). Grundlage der Vererbung.
- Methoden klassisch via `Foo.prototype.m = function(){...}`. Vererbung: `Parent.call(this, ...)` + `Object.setPrototypeOf(Child.prototype, Parent.prototype)` FOLIE schreibt `Object.setPrototypeOf` — korrekt: `setPrototypeOf`.

```
class Person {
  constructor(name){ this.name = name; }
  vorstellen(){ console.log("Hallo, " + this.name); }
}
```

```
class Dozent extends Person {
  constructor(name, fach){ super(name); this.fach = fach; }
}
new Dozent("Mikka", "WE").vorstellen(); // "Hallo, Mikka"
```

Fallen / Bugs

- **Falle:** `class` ist syntaktischer Zucker über Prototypes — Verhalten identisch, lesbarer.
- **Falle:** `super(...)` im Subklassen-Konstruktor VOR `this`-Nutzung aufrufen.

Drill

- [Erstellen] Klasse `Tier` mit Konstruktor `name` und Methode `laut()`.

Lösung: `class Tier{ constructor(n){this.name=n;} laut(){...} }`

J11 — JSON & Browser-APIs (Produktion, mittel)

- **JSON:** Datenaustauschformat. `JSON.parse(str)` → Objekt; `JSON.stringify(obj)` → String.
- **JSON-Typen:** Objekt `{}`, Array `[]`, String `"..."`, Number, `true` / `false`, `null`. **KEINE** Funktionen, Prototypen, zirkulären Referenzen.
- **Deep-Clone-Trick:** `JSON.parse(JSON.stringify(obj))` (kein Zyklus erlaubt; modern: `structuredClone()`).
- **Standardobjekte:** `window` (`innerWidth/innerHeight`), `document` (`cookie`, `title`), `Date` (`new Date()`).
- **Standardfunktionen:** `alert(msg)`, `encodeURIComponent` / `decodeURI` FOLIE schreibt `encode/decode` — korrekt `encodeURIComponent` / `decodeURI` (bzw. `encodeURIComponent`), `isFinite`, `isNaN`, `parseFloat`, `parseInt`.
- **String:** `s.length`, `s.charAt(0)` FOLIE schreibt `z.charAt(0)` — korrekt `charAt` (oder `z[0]`).

Fallen / Bugs

- **Falle:** `JSON.stringify` auf Objekt mit Funktion → Funktion fällt weg; mit Zyklus → `TypeError`.
- **Falle:** `JSON.parse` von ungültigem JSON → `SyntaxError`.

Drill

- [Erstellen] Objekt `{x:5}` in JSON-String und zurück.

Lösung: `JSON.stringify({x:5})` → `'{"x":5}'`; `JSON.parse('{"x":5}')`.x → 5.

Zusatz (niedrig-Prio, im JS-Lehrplan erwähnt)

- **Prototype-Chain:** `Object.getPrototypeOf(o)` läuft bis `null`.
- **WebSocket:** bidirektional über TCP, Handshake via HTTP, URL-Schema `ws` / `wss` (Standard-Port 80/443). FOLIE-Beispiel `new WebSocket("http://...")` falsch → korrekt `ws://` / `wss://`. Öffnen ist asynchron (`socket.readyState`, `OPEN==1` Folie "OPEN == 1" = OCR-Müll); `onopen/onmessage/onerror/onclose`, Daten in `event.data`.
- **Umfeld:** TypeScript (Superset mit Typen → kompiliert zu JS), WASM, GraphQL.